

Automated Deployment Pipeline

SkyBridge Network Recovery Optimizer

Student name(s)	
Course / section	[Enter course and section]
Submission date	[Enter date]
GitHub repository	https://github.com/MayFairMI6/skybridge-network-recovery-optimizer

Completed requirements

Requirement	Verification
Git/GitHub	Complete. The repository contains the application, Dockerfile, Jenkinsfile, Terraform configuration, Jenkins Docker setup, and documentation.
Automated CI Trigger	Complete. SCM polling H/2 * * * * detected a GitHub push and automatically started successful Jenkins build #46; Build Now was not used.
Jenkins (Builder)	Complete. Jenkins checks out the repository, builds a Docker image, and runs terraform apply.
Terraform (Deployer)	Complete. Terraform uses the kreuzwerker/docker provider and the local Docker socket to create or replace the application container.
Docker (Runtime)	Complete. Docker Desktop hosts both Jenkins and the deployed SkyBridge application.

Local setup and verification

The project runs locally on macOS using Docker Desktop. Jenkins runs as the local-pipeline-jenkins Docker container and is configured in a Docker-outside-of-Docker pattern: the Docker Desktop socket is mounted into Jenkins so the Jenkins Docker CLI and Terraform Docker provider can manage the host daemon.

The deployed SkyBridge dashboard is a stochastic airline disruption-recovery simulation. It evaluates flight holds, cancellation/rebooking, hotel protection, recovery inventory, passenger priority, airport-pair slot pressure, weather, airspace, and volcanic-ash signals. It is a classroom prototype using synthetic network and passenger data, not a production airline system.

Verification sequence: start Docker Desktop; run Docker Compose from the repository root; open Jenkins at <http://localhost:8080>; run the Pipeline job; confirm a successful console log; verify local-pipeline-app is deployed to port 8081; then open <http://localhost:8081> and capture the running dashboard.

Captured verification evidence

The following pages contain unedited screenshots captured from the local environment. They show successful Jenkins builds, Terraform deployment, Docker runtime evidence, and the deployed application.

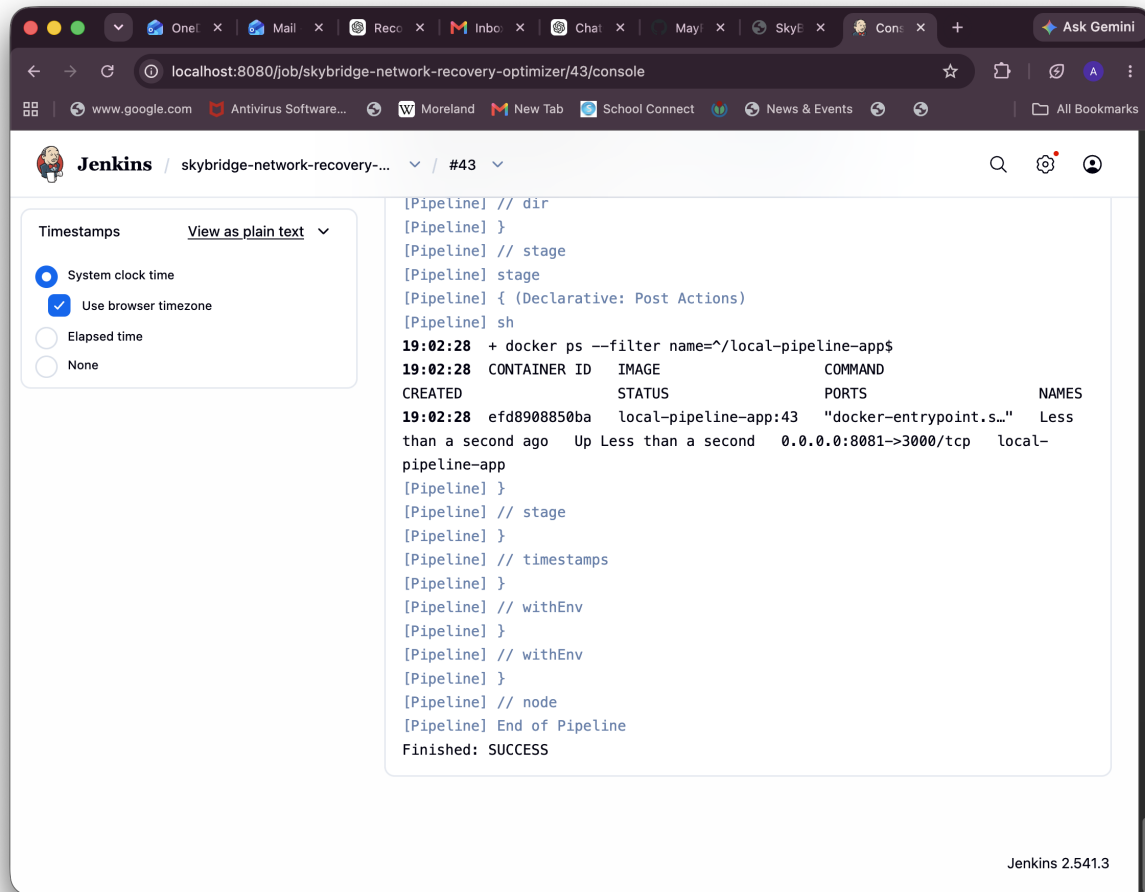
1. Jenkins build history

The screenshot shows the Jenkins web interface at localhost:8080/view/all/builds. The left sidebar contains a 'New Item' button, a 'Build History' button, a 'Build Queue' section (showing 'No builds in the queue.'), and a 'Build Executor Status' section (showing '0 of 2 executors busy'). The main area is titled 'Build History of Jenkins' and displays a table of build records.

S	Build	Time Since	↑	Status	
✓	skybridge-network-recovery-optimizer #43	2 min 22 sec		stable	[icon]
✓	skybridge-network-recovery-optimizer #42	2 min 35 sec		stable	[icon]
✓	skybridge-network-recovery-optimizer #41	7 min 46 sec		stable	[icon]
✓	skybridge-network-recovery-optimizer #40	12 min		stable	[icon]
✓	skybridge-network-recovery-optimizer #39	12 min		stable	[icon]
✓	skybridge-network-recovery-optimizer #38	16 min		stable	[icon]
✓	skybridge-network-recovery-optimizer #37	18 min		stable	[icon]
✓	skybridge-network-recovery-optimizer #36	22 min		stable	[icon]
✓	skybridge-network-recovery-optimizer #35	22 min		stable	[icon]

Successful Jenkins build history for the SkyBridge Pipeline job.

2. Jenkins console output



The screenshot shows the Jenkins web interface at localhost:8080. The console output for job 'skybridge-network-recovery-optimizer/43' is displayed. The output shows the pipeline execution steps, including directory listing, stage declaration, and a successful completion message.

```
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
19:02:28 + docker ps --filter name=~/local-pipeline-app$
19:02:28 CONTAINER ID   IMAGE                                COMMAND
CREATED              STATUS                PORTS                NAMES
19:02:28 efd8908850ba        local-pipeline-app:43              "docker-entrypoint.s..." Less
than a second ago   Up Less than a second   0.0.0.0:8081->3000/tcp   local-
pipeline-app
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.541.3

Terraform created local-pipeline-app:43, published port 8081, and the pipeline finished successfully.

3. Docker runtime

```
referenced-chatgpt-conversation-this-is-untrusted-4 — zsh — 80x24

=> => naming to docker.io/library/referenced-chatgpt-conversation-this-i 0.0s
=> => unpacking to docker.io/library/referenced-chatgpt-conversation-thi 0.0s
=> resolving provenance for metadata file 0.0s
[+] Building 1/1
  ✓ referenced-chatgpt-conversation-this-is-untrusted-4-jenkins Built 0.0s
anony@USF-Wifi-134-102-33 referenced-chatgpt-conversation-this-is-untrusted-4 %
[docker compose up -d --force-recreate jenkins]
[+] Running 1/1
  ✓ Container local-pipeline-jenkins Started 0.5s
anony@USF-Wifi-134-102-33 referenced-chatgpt-conversation-this-is-untrusted-4 %
[docker ps]
CONTAINER ID   IMAGE                                STATUS      PORTS                               NAMES
MAND          CREATED          STATUS      PORTS                               NAMES
8f74186dae2f   referenced-chatgpt-conversation-this-is-untrusted-4-jenkins   "u
sr/bin/tini -- /u..."   8 seconds ago   Up 7 seconds   0.0.0.0:8080->8080/tcp, [
:]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   local-pipel
ine-jenkins
ff8ac9a04c5f   seed-image-mysql-sqli                                "do
cker-entrypoint.s..."   7 months ago   Up 17 minutes   3306/tcp, 33060/tcp
mysql-10.9.
0.6
anony@USF-Wifi-134-102-33 referenced-chatgpt-conversation-this-is-untrusted-4 %
```

Docker Compose starts local-pipeline-jenkins; docker ps confirms Jenkins is running on ports 8080 and 50000.

4. Running deployed application

The screenshot displays the SkyBridge Network Recovery Optimizer web application. The browser address bar shows `localhost:8081`. The application interface includes a flight path selector at the top with buttons for SFO, DXB (constrained hub), LHR, and DEL. Below this, a text block explains the optimizer's evaluation of weather events and hub capacity. A green status bar indicates weather conditions: Open-Meteo: 35° · gusts 8 mph · visibility 18 km · cloud 90% · precipitation 0 mm; Automatic forecast: weather 35% · news/airspace 0% · volcanic ash 0% — dominant risk: none; and Airspace monitor: no material closure or rerouting signal for DXB. Three summary boxes show: Affected flights: 6, Protected itineraries: 14, and High-risk connections: 12. On the right, there are sliders for Risk aversion (55% tail-risk weight) and Weather severity (11% multi-variable weather risk). Below these are dropdowns for Disrupted legs (LHR-DEL, SIN-HKG, SYD-AKL), Route origin (SFO), Connection after hub (LHR), and Final destination (DEL). A prominent blue button reads "Run stochastic recovery optimization". Below the button, a note states: "Every run resamples weather duration, connection failures, recovery-seat availability, cancellations, reroutes, and new-demand revenue." At the bottom, a "Network action recommendation" section highlights "Recommend: Hold 90 minutes" with a brief explanation. Below this is a table comparing candidate policies.

CANDIDATE POLICY	EXPECTED COST	95TH PERCENTILE	RISK-ADJUSTED SCORE	PROTECTED ITINERARIES	DECISION
Hold 90 minutes	\$45,081	\$61,780	\$56,657	39.5 / 46	Recommended
Hybrid: protect long-haul, rebook others	\$73,171	\$85,750	\$81,983	29.7 / 46	Alternative
Cancel + same-day rebook	\$80,967	\$94,862	\$89,861	20.7 / 46	Alternative

SkyBridge Network Recovery Optimizer running locally at `http://localhost:8081`.